

DQ Compass

Reporting & Analytics — Metrics Reference

MBA-ESG / SG GSC Datathon 2026 — *DQ Compass: a Plug-and-Play Data Quality Control Layer for EUCs*

Working Group 1

Generated 2026-09-08

Every KPI, panel and chart on the **/reporting** page, and the exact calculation behind each. Source of truth:

Concern	File / function
Page assembly (what each panel receives)	rule_authoring/app.py :: reporting_context()
Score / dataset score / summaries / alerts / trend	reporting/scorecard.py
Per-control metrics (null_count, invalid_count, ...)	controls.py
Threshold flip, exception_pct, ledger counts	dq_engine.py
Chart geometry (score bar, trend line, bars, sparkline)	rule_authoring/charts.py
Page markup / labels	rule_authoring/templates/reporting.html

1 How the page is fed

The page never re-reads the raw datasets. It reads the **Evidence Pack** that `dq_engine.py` wrote for each run under `evidence/<run_id>/` (per-rule `*_result.json`, `*_exceptions.csv`, `run_summary.json`) plus the append-only `evidence/runs_ledger.jsonl`.

Focus run, previous run, window

`reporting_context(focus_run_id)` lists every run (**newest first**) and picks a **focus run**:

- **focus run** = the run chosen in the selector (`?run=<id>`); default = the most recent run. Drives *every panel* except the two below.
- **previous run** = the run immediately older than the focus run (`runs[focus_idx + 1]`). Used by the score delta and “What changed”.
- **window** = the focus run plus up to 11 older runs (`runs[focus_idx : focus_idx + 12]`). Used only by the **Trend chart** and the **Rule reliability table**.

If no run is selected and there is only one run, the delta / previous-run panels show “first run”. An unknown `?run` id silently falls back to the latest run.

Vocabulary used throughout

Term	Meaning
status	PASS / FAIL / ERROR for one rule in one run (from <code>*_result.json</code>).
ERROR	The rule could not execute: unknown control type, or its data source is missing / retired. The run continues; the rule scores as a total miss.
exception_count	Number of rows in that rule's <code>_exceptions.csv</code> = the failing records. Kept even when a threshold flips the status to PASS.
exc_pct	<code>metrics.exception_pct</code> if the engine recorded it, else <code>round(100 x exception_count / total_records, 2)</code> , else 100 if FAIL else 0.
W(severity)	Severity weight: High = 5, Medium = 2, Low = 1.
counts	Per-run tally {PASS, FAIL, ERROR} stored in the run ledger by the engine.

2 Shared building blocks

2.1 Per-control metrics (controls.py)

Each control function returns {status, metrics, exceptions}. total_records is the row count actually evaluated. status = FAIL as soon as one exception exists (before any threshold is applied).

Dimension	Exceptions = rows where...	Headline metric
Completeness	field IS NULL or trimmed value is empty	completeness_pct = round(100 x (total - null_count) / total, 2)
Validity	field NOT IN allowed_values / NOT full-match of regex / outside [min,max]	valid_pct = round(100 x (total - invalid_count) / total, 2)
Uniqueness	row is part of any duplicated group on the key column(s) (pandas duplicated(keep=False))	duplicate_pct = round(100 x duplicate_count / total, 2)
Consistency	field value not found in the set of ref_dataset.ref_field values	orphan_pct = round(100 x orphan_count / total, 2)
Timeliness	(reference_date - field_date).days > max_lag_days (exceptions get an added lag_days column)	within_sla_pct = round(100 x (total - sla_breach_count) / total, 2)
Reconciliation	key missing on one side, OR abs(src - ref) / ref x 100 > tolerance_pct (total = number of merged/joined rows)	reconciled_pct = round(100 x (total - break_count) / total, 2)

2.2 Tolerance threshold (dq_engine.py :: _apply_threshold)

If a rule sets threshold_pct, the engine records exception_pct = round(100 x exception_count / total_records, 2) and, when the rule FAILED but exception_pct <= threshold_pct, flips the status to **PASS** and sets within_threshold = true. The exception rows are still written and still counted — only the status changes.

2.3 Composite DQ score (reporting/scorecard.py :: overall_dq_score)

One number for a whole run, 0–100, higher is better. Severity-weighted so a High failure hurts 5x a Low one.

```
weight = SUM over every rule in the run of W(severity)
penalty = SUM over every rule of:
PASS -> 0
ERROR -> W(severity) (counted as a total miss)
FAIL -> W(severity) x min(1, exc_pct / 100)

score = 100 if weight == 0
= max(0, round(100 x (1 - penalty/weight), 1)) otherwise
```

A FAIL with a tiny exception rate barely moves the score; a FAIL at 100% exceptions, or an ERROR, costs the rule's full weight. build_dataset_score uses the identical formula restricted to one dataset's rules.

2.4 Traffic light (used in the CSV / HTML downloads)

status / severity	Light
status == PASS	GREEN
status == ERROR	GREY
status == FAIL and severity == High	RED
status == FAIL and severity != High	ORANGE

3 The KPI strip (top of the page)

Six tiles, all computed for the **focus run**.

3.1 Composite DQ score

- **Value:** `overall_dq_score(focus_run)` — section 2.3.
- **Tile colour:** green if score ≥ 90 , amber if ≥ 70 , red otherwise.
- **Delta:** `round(score(focus) - score(previous_run), 1)`. Shown as a triangle up (green) or down (red); “first run” if there is no older run.
- **Sparkline:** `score_series` = the composite score of each run in the window, oldest to newest (section 6.1).

3.2 Rules passing

- **Value:** `pass_rate = round(100 x n_pass / n_rules, 1) %`.
- `n_pass` = rules whose status is PASS in the focus run (a threshold flip counts as PASS). `n_rules` = number of rule results in the run.
- **Foot:** “`n_pass` of `n_rules` controls”.

3.3 Open High-severity issues

- **Value:** `len(alerts)`.
- `alerts (build_alerts)` = every rule in the focus run with status in `{FAIL, ERROR}` AND `severity == High`.
- Tile is red if any exist, green if none.

3.4 Exception records

- **Value:** `total_exc = SUM of exception_count` over every rule in the focus run.
- This is a count of *failing rows* across all rules, not a count of rules.

3.5 Datasets under control

- **Value:** `covered / datasets_total`.
- `datasets_total` = number of registered data sources.
- `covered` = how many of them have at least one *active* rule on at least one dimension (from `build_coverage: {source: {dimension: [rule_ids]}}`, count sources with a non-empty cell).

3.6 Failing / errored

- **Value:** `n_fail / n_error`, read straight from the focus run's ledger counts (FAIL, ERROR).
- “errored” = a rule whose source or config is broken; investigate before trusting the rest of the run.

4 “For control owners & risk”

4.1 Overall data quality (score bar)

Same number as KPI 3.1, drawn as a bar (`charts.score_bar`): fill width = score %, colour band at 90 / 70, tick marks at 70 and 90, scale 0-70-90-100. The delta pill repeats the vs-previous-run change.

4.2 Trend — last N runs (`charts.trend_chart`)

Input = `ctx.trend`: the window runs, oldest to newest, each {label = MM-DD, pass, fail, error, score} where pass/fail/error come from that run's ledger counts and score = `overall_dq_score` of that run.

- **Line** (accent colour): the composite DQ score per run.
- **Y axis is auto-floored** so movement stays visible when scores cluster high: `lo = max(0, min(score) - 8)`; then `lo = 0 if lo < 12 else round(lo/5) x 5`; the axis runs `lo..100`.
- **Composition strip** under each run: a small stacked bar split pass / fail / error (green / red / grey), each segment width proportional to its share of that run's total.
- Hovering a point shows timestamp - nP/nF/nE - score.

4.3 Dataset health (`build_dataset_score`)

One row per dataset that has rules, **sorted worst score first**.

```
for each dataset (grouped by rule_configuration_snapshot.dataset_scope):
  weight = SUM W(severity) over that dataset's rules
  penalty = SUM PASS 0 | ERROR W | FAIL W x min(1, exc_pct/100)
  dq_score = 100 if weight == 0
  else max(0, round(100 x (1 - penalty/weight), 1))
  n_pass / n_fail / n_error = status tallies ; n_rules = count
```

Bar width = `dq_score %`. Colour band 90 (green) / 70 (amber) / below (red). The green / red / err chips are `n_pass / n_fail / n_error`.

4.4 Coverage & status (3-state matrix)

Rows = data sources, columns = the 6 DQ dimensions. Each cell:

```
ids = ACTIVE rules with (dataset_scope == row, control_type == column)
state = 'none' if ids is empty -> grey, blank
= 'fail' if any of those rules is FAIL or ERROR -> red, shows count
= 'pass' otherwise -> green, shows count
```

The number in a coloured cell is how many rules sit in that dataset x dimension. “Datasets under control” (KPI 3.5) = rows with at least one non-grey cell.

4.5 Open issues — failing High-severity controls

The alerts list from 3.3, one card each: rule id + name, exception_count and dataset, owner, and the remediation_action text looked up from `control_catalogue.csv` by rule id.

4.6 Exceptions by owner (accountability)

Horizontal bars (`charts.hbars`). Input = `build_exception_summary` rows for dimension owner: for each owner, `total_exceptions = SUM exception_count` over that owner's rules. Sorted by total descending; each bar width = `value / (largest value) x 100%`.

5 “For data engineers”

5.1 Exceptions by DQ dimension

Same as 4.6 but grouped by `control_type` (Completeness, Validity, ...). Answers “which kind of quality problem produces the most failing rows in this run”.

5.2 Top offending rules (this run)

`by_rule` = every rule in the focus run with `exception_count > 0`, sorted descending, top 8, drawn with hbars.
Bar value = that rule's `exception_count`.

5.3 Rule reliability (last N runs)

Aggregated over the **window** (focus run + up to 11 older).

```
for each rule seen in the window:
runs = number of window runs that contain the rule
fail = number of those runs where status == FAIL
error = number where status == ERROR
last = the rule's status in the focus run

fail rate = (fail + error) / max(1, runs) -> table is sorted by this, desc; top 12
mini-bar width = (fail + error) / runs x 100%
```

5.4 What changed since the previous run

Compares each rule present in **both** the focus run and the previous run:

```
status changed -> flip: from -> to (green up if new status is PASS)
both FAIL, exception_count changed
-> delta: (cur - prev) exception(s) (green if fewer)
```

“No previous run to compare against” on the oldest run; “no control changed” when both runs agree.

5.5 Broken controls (ERROR)

`error_rules` = rules whose status is ERROR in the focus run, with the recorded reason (e.g. *source unavailable*).
These are excluded from a meaningful pass rate and cost full weight in the score.

6 Chart encodings (rule_authoring/charts.py)

All charts are inline SVG/HTML with no library; colours are CSS variables so they follow the light/dark theme.

Chart	Encoding
score_bar(value, delta)	Fill width = clamp(value, 0, 100) %. Colour: >=90 ok, >=70 warn, else bad. Fixed ticks at 70 and 90; scale labels 0 / 70 / 90 / 100.
sparkline(vals)	Polyline in a 60x16 box. $y = 16 - (v - \min) / (\max - \min) \times 13$; x evenly spaced. Needs >= 2 points, else nothing.
trend_chart(series)	See 4.2. Score line + faint area; per-run pass/fail/error strip; auto-floored y axis; gridlines at {lo, midpoint rounded to 5, 100}.
hbars(items, label, val, tone)	One row per item with val > 0. Bar width = val / max(val) x 100%. Rows keep the input order (callers pre-sort by value desc).
mini_bar(hits, total, tone)	Single inline bar, width = hits / total x 100%.
donut(p, f, e)	Stacked ring; each arc length proportional to p, f, e over their sum; centre shows p+f+e. (Defined for reuse; the run-detail page uses it.)

7 Downloads

The links at the bottom of the page serve files that `reporting/scorecard.py` writes into `reporting/`. They are regenerated by `dqcompass report` (or `python reporting/scorecard.py <run_id>` for a specific run).

File	Builder	One row per / contents
scorecard.csv	build_scorecard	one rule: status, traffic_light, exception_count, and every metric_* value
dataset_score.csv	build_dataset_score	one dataset: n_pass/fail/error, dq_score (section 4.3)
coverage.csv	build_coverage	one dataset x dimension pair actually used: rule_ids, n_rules
exception_summary.csv	build_exception_summary	long format: dimension in {severity, owner, control_type, dataset} x key -> failing_rules, error_rules, total_exceptions
trend.csv	build_trend	one run from the ledger: rules_executed, pass, fail, error
alerts.json	build_alerts	failing High-severity rules + remediation_action
exceptions_detail.csv	build_exceptions_detail	every failing row from every rule, prefixed with rule_id + severity
scorecard.html	to_html	standalone traffic-light table for a quick demo

Note: the downloadable files reflect whichever run `scorecard.py` last ran for (the latest run by default) - they are not re-scoped by the on-page run selector. Pass a run id on the command line to export a past run.

Worked example (current seeded demo, focus = latest run)

KPI	Value	Why
Composite DQ score	87.2 / 100	14 rules; weights sum = 46; penalty ~ 5.9 (High FAILs at high exc_pct + Medium FAILs); $100 \times (1 - 5.9/46) \sim 87.2$
Rules passing	57.1 %	8 PASS of 14
Open High-severity issues	4	VAL_LIST_FAIL, CON_FAIL, TIM_FAIL, REC_FAIL
Exception records	~74k	sum of exception_count (CMP_FAIL 17,708 + UNQ_FAIL 27,290 + CON_FAIL 8,997 + CMP_THRESHOLD_PASS 19,969 + ...)
Datasets under control	3 / 6	finance_data, feed_status, recon_euc_positions have rules; the 3 reference-only sources do not
Failing / errored	6 / 0	ledger counts for the run